

MetaDAO Futarchy AMM

Smart Contract Security Assessment

September 2025

Prepared for:

MetaDAO

Prepared by:

Offside Labs

Gongyu Shi

Ripples Wen





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Incorrect Output Amount Calculation During Swap	6
4.2	Conditional Tokens in AMM Vaults Remain Unredeemed After Proposal Finaliza- tion	7
4.3	Swap Conditional Tokens of Finalized Proposal Is Allowed in conditional_swap	9
4.4	Missing Minimum Liquidity Check When Launching Proposal	10
4.5	Incorrect Protocol Profit Calculation in Arbitrage After Conditional Swaps . . .	10
4.6	Liquidity Provided During DAO Initialization Is Not Tracked in a Position	12
4.7	Incorrect FutarchyAmm Initialization When No Liquidity is Provided During DAO Creation	13
4.8	Missing Validation for description_url During Proposal Initialization	14
4.9	Malicious Proposals Can Block New Proposals in DAO	15
4.10	AMM Position Account Cannot Be Closed	16
4.11	Informational and Undetermined Issues	16
5	Disclaimer	18



1 About Offside Labs

Offside Labs stands as a pre-eminent security research team, comprising highly skilled hackers with top - tier talent from both academia and industry.

The team demonstrates extensive and diverse expertise in modern software systems, which encompasses yet are not restricted to *browsers, operating systems, IoT devices, and hypervisors*. Offside Labs is at the forefront of innovative domains such as *cryptocurrencies and blockchain technologies*. The team achieved notable accomplishments including the successful execution of remote jailbreaks on devices like the **iPhone** and **PlayStation 4**, as well as the identification and resolution of critical vulnerabilities within the **Tron Network**.

Offside Labs actively involves in and keeps contributing to the security community. The team was the winner and co-organizer for the *DEFCON CTF*, the most renowned CTF competition in Web2. The team also triumphed in the **Paradigm CTF 2023** in Web3. Meanwhile, the team has been conducting responsible disclosure of numerous vulnerabilities to leading technology companies, including *Apple, Google, and Microsoft*, safeguarding digital assets with an estimated value exceeding **\$300 million**.

During the transition to Web3, Offside Labs has attained remarkable success. The team has earned over **\$9 million** in bug bounties, and **three** of its innovative techniques were acknowledged as being among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



2 Executive Summary

Introduction

Offside Labs completed a security audit of *MetaDAO Futarchy AMM* smart contracts, starting on August 18th, 2025, and concluding on September 3rd, 2025.

Project Overview

The MetaDAO Futarchy AMM integrates the functionality of the amm program into the autocrat program. Specifically, each DAO now has its own AMM. Users can swap the DAO's native token pair (e.g., META/USDC) at any time. Once a proposal is launched, they can seamlessly swap the proposal's conditional token pairs (e.g., pMETA/pUSDC and fMETA/fUSDC). And an automatic arbitrage logic is applied to all swaps within the AMM.

Audit Scope

The assessment scope contains mainly the smart contracts of the autocrat program for the *MetaDAO Futarchy AMM* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- MetaDAO Futarchy AMM
 - Codebase: <https://github.com/metaDAOproject/programs/>
 - PR-321:
 - Commit Hash: fb05449e167fbe189ede276f47dad5c2b03df23e
 - Link: [PR-321](#)

We listed the files we have audited below:

- MetaDAO Futarchy AMM
 - programs/autocrat/src/instructions/spot_swap.rs
 - programs/autocrat/src/instructions/initialize_proposal.rs
 - programs/autocrat/src/instructions/conditional_swap.rs
 - programs/autocrat/src/instructions/initialize_futarchy_amm.rs
 - programs/autocrat/src/instructions/collect_fees.rs
 - programs/autocrat/src/instructions/withdraw_liquidity.rs
 - programs/autocrat/src/instructions/finalize_proposal.rs
 - programs/autocrat/src/instructions/provide_liquidity.rs
 - programs/autocrat/src/instructions/initialize_dao.rs
 - programs/autocrat/src/instructions/update_dao.rs
 - programs/autocrat/src/state/dao.rs
 - programs/autocrat/src/state/proposal.rs
 - programs/autocrat/src/state/futarchy_amm.rs
 - programs/autocrat/src/state/amm_position.rs
 - programs/autocrat/src/lib.rs



Findings

The security audit revealed:

- 2 critical issues
- 1 high issue
- 3 medium issues
- 4 low issues
- 2 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Incorrect Output Amount Calculation During Swap	Critical	Fixed
02	Conditional Tokens in AMM Vaults Remain Unredeemed After Proposal Finalization	Critical	Fixed
03	Swap Conditional Tokens of Finalized Proposal Is Allowed in conditional_swap	High	Fixed
04	Missing Minimum Liquidity Check When Launching Proposal	Medium	Fixed
05	Incorrect Protocol Profit Calculation in Arbitrage After Conditional Swaps	Medium	Fixed
06	Liquidity Provided During DAO Initialization Is Not Tracked in a Position	Medium	Fixed
07	Incorrect FutarchyAmm Initialization When No Liquidity is Provided During DAO Creation	Low	Fixed
08	Missing Validation for description_url During Proposal Initialization	Low	Fixed
09	Malicious Proposals Can Block New Proposals in DAO	Low	Acknowledged
10	AMM Position Account Cannot Be Closed	Low	Acknowledged
11	Missing Validation of slots_per_proposal When Updating DAO	Informational	Fixed
12	Redundant Arbitrage Calculations in Trivial Cases	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Incorrect Output Amount Calculation During Swap

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Math Error

Description

When a swap occurs, the pool calculates the expected amount of output tokens in the `swap` function. The invariant is that the product of the reserves remains constant:

$$(I + \delta_I) \cdot (O - \delta_O) = I \cdot O$$

where

- I is the input reserve
- O is the output reserve
- δ_I is the input amount
- δ_O is the output amount

And δ_O should be calculated as

$$\delta_O = \frac{\delta_I \cdot O}{I + \delta_I}$$

However, in the current implementation, the input reserve I in the denominator is scaled by `MAX_BPS` incorrectly.

```
373 let input_amount_with_lp_fee = input_amount_after_protocol_fee as u128 *  
    (MAX_BPS - LP_TAKER_FEE_BPS) as u128 / MAX_BPS as u128;  
374  
375 let numerator = input_amount_with_lp_fee * output_reserve as u128;  
376  
377 let denominator = (input_reserve as u128 * MAX_BPS as u128) +  
    input_amount_with_lp_fee as u128;  
378  
379 let output_amount = (numerator / denominator) as u64;
```

[programs/autocrat/src/state/futarchy_amm.rs#L373-L379](#)

Impact

Because the denominator is significantly larger than it should be, users receive far fewer output tokens than expected. The missing tokens remain in the pool, directly benefiting liquidity providers.



Recommendation

Remove the early division by `MAX_BPS` when calculating `input_amount_with_lp_fee`.

Mitigation Review Log

Fixed in commit `ddc508539e41bd55326646b6344a8f3c0cfdebba`.

4.2 Conditional Tokens in AMM Vaults Remain Unredeemed After Proposal Finalization

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When finalizing a proposal, the reserves of both the quote and base tokens in the AMM are updated according to the proposal outcome:

```
124     if new_proposal_state == ProposalState::Passed {
125         ...
126         spot.base_reserves += pass.base_reserves;
127         spot.quote_reserves += pass.quote_reserves;
128         spot.base_protocol_fee_balance +=
129     ↪ pass.base_protocol_fee_balance;
129         spot.quote_protocol_fee_balance +=
130     ↪ pass.quote_protocol_fee_balance;
130     } else {
131         spot.base_reserves += fail.base_reserves;
132         spot.quote_reserves += fail.quote_reserves;
133         spot.base_protocol_fee_balance +=
134     ↪ fail.base_protocol_fee_balance;
134         spot.quote_protocol_fee_balance +=
135     ↪ fail.quote_protocol_fee_balance;
135     }
```

[programs/autocrat/src/instructions/finalize_proposal.rs#L124-L135](#)

At this point, some conditional tokens may still remain in the conditional AMM vaults. These tokens should be redeemed back into META or USDC and transferred into the corresponding native AMM vaults, but this step is missing in the current implementation.



Impact

As conditional tokens are not redeemed, the recorded reserve amounts may not match the actual balances in the AMM vaults. In particular, the reserves recorded in state are over-estimated. As a result, when liquidity providers attempt to withdraw funds, the operation may fail due to insufficient tokens in the vaults, which leads to a DoS condition.

The discrepancy between internal accounting and actual token balances will continue to grow with each finalized proposal, compounding the risk over time.

Proof of Concept

Suppose there are no fees and no arbitrage in the AMM pools. Initially, the quote and base reserves are zero for all three pools (META/USDC, pMETA/pUSDC, fMETA/fUSDC), and all four conditional AMM vaults also start with zero balance.

1. A user adds 2000 META and 2000 USDC as liquidity to the DAO AMM. The META and USDC AMM vaults now each hold 2000 tokens.
2. A proposal of the DAO is launched. The initial reserves are set to 1000:1000 for all three pools (META/USDC, pMETA/pUSDC, fMETA/fUSDC).
3. The user performs a conditional swap with 1000 pUSDC. The expected output is 500 pMETA. The pMETA/pUSDC pool reserve is updated to 500:2000.
4. 1000 pUSDC is transferred into the pUSDC AMM vault.
5. A CPI call to `split_tokens` is executed, 500 META is split into 500 pMETA and 500 fMETA.
6. 500 pMETA is transferred to the user, completing the swap.
7. The proposal is finalized with the outcome `Passed`. The reserves are updated as follows:

- META reserve: $1000 \text{ (META)} + 500 \text{ (pMETA)} = 1500$
- USDC reserve: $1000 \text{ (USDC)} + 2000 \text{ (pUSDC)} = 3000$

However, the actual vault balances are:

- META vault: $2000 - 500 = 1500$ (500 are used for splitting)
 - USDC vault: 1000
 - fMETA vault: 500
 - pUSDC vault: 2000
8. The user withdraws all liquidity, they expect to receive 1500 META and 3000 USDC based on the current META and USDC reserves. However, there are only 1000 USDC tokens in the AMM vault, the withdrawal request is failed.

The 2000 pUSDC should be redeemed in Step 7.

Recommendation

Add a CPI to `redeem_tokens` to redeem the conditional tokens left in the vaults when finalizing proposals.



Mitigation Review Log

Fixed in commit c5eaf3282737c0938107dec9823d6bb193a12b6d.

4.3 Swap Conditional Tokens of Finalized Proposal Is Allowed in conditional_swap

Severity: High

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

The `conditional_swap` IX allows a swap in one of the conditional pools. However, there is no validation for the status of the `proposal` account.

```
22  #[account(  
23      has_one = dao, has_one = base_vault, has_one = quote_vault,  
24      has_one = pass_base_mint, has_one = pass_quote_mint,  
25      has_one = fail_base_mint, has_one = fail_quote_mint,  
26      has_one = question  
27  )]  
28  pub proposal: Box<Account<'info, Proposal>>,  
  
    programs/autocrat/src/instructions/conditional_swap.rs#L22-L28
```

If the proposal has already been finalized, it is still possible to make swap of its conditional tokens.

Impact

If the provided proposal has been finalized, the swap of its conditional tokens still affect the TWAP of the current conditional AMM pools, which will directly determine the result of the current proposal.

Recommendation

Add a check to ensure the state of `proposal` is `Pending` in `conditional_swap`.

Mitigation Review Log

Fixed in commit c25493d742c4094446b6e72b4f6e6df564a4fc90.



4.4 Missing Minimum Liquidity Check When Launching Proposal

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

When a proposal is launched, the `futarchy_amm` associated with the DAO transitions from the `Spot` state to the `Futarchy` state. In the `Futarchy` state, adding or removing AMM liquidity is disallowed. Therefore, it is critical to ensure sufficient liquidity exists in the AMM at the time of launching the proposal. Currently, this validation is missing.

Impact

If the AMM reserves are too small, the conditional pools can be easily manipulated, potentially influencing the outcome of the proposal.

Recommendation

Before launching a proposal, ensure that both the quote and base token reserves satisfy the minimum thresholds defined by `min_quote_futarchic_liquidity` and `min_base_futarchic_liquidity`.

Mitigation Review Log

Fixed in commit 5069dd53ea3ac953d01bfff51de201f0d9f3465.

4.5 Incorrect Protocol Profit Calculation in Arbitrage After Conditional Swaps

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

After a swap in a conditional pool, the function `arbitrage_after_conditional_swap` is called to perform arbitrage.



```
641 let final_pass_output = pass
642   .feeless_swap(best_arb_input_amount, conditional_direction)
643   .unwrap();
644 let final_fail_output = fail
645   .feeless_swap(best_arb_input_amount, conditional_direction)
646   .unwrap();
647 let final_conditional_output = std::cmp::min(final_pass_output,
648   final_fail_output);
649 let final_spot_output = spot.feeless_swap(final_conditional_output,
650   spot_direction).unwrap();
```

[programs/autocrat/src/state/futarchy_amm.rs#L641-L649](#)

Here, `final_spot_output` represents the maximum arbitrage profit denominated in the native token (META or USDC). This profit can be split equally into both conditional tokens. One of them should be credited to the user as arbitrage rewards, while the other should be accumulated as protocol fee. However, the current implementation never adds this portion into the protocol fee balance.

Impact

Each conditional swap leads to a systematic loss of protocol fees in conditional tokens. The unaccounted tokens remain within pool liquidity and may incidentally benefit liquidity providers.

Proof of Concept

Suppose there are no fees during swap and the current reserve of the pools META/USDC, pMETA/pUSDC, fMETA/fUSDC are 2000:500, 1000:1000, 1000:1000 respectively.

1. A user swaps 1000 pUSDC for pMETA. The expected amount of output pMETA is 500 (`conditional_output`).
2. Arbitrage simulation determines that selling all 500 (`best_arb_input_amount`) pMETA maximizes profit.
3. 500 pMETA are sold for 1000 pUSDC, while 500 fMETA are borrowed and sold for 333 fUSDC.
4. 333 fUSDC and 333 pUSDC are merged to 333 USDC, which can buy 799 (`final_spot_output`) META. There are $1000 - 333 = 667$ pUSDC remained.
5. 799 META is split to 799 pMETA and 799 fMETA.
6. Remaining 667 pUSDC is swapped for 400 (`pass_profit`) pMETA.
7. The arbitrage function returns the result:
 - `spot_profit` is `final_spot_output` - `best_arb_input_amount` = $799 - 500 = 299$
 - `pass_profit` is 400
 - `fail_profit` is 0



8. The `fail_profit` is added to `fail.base_protocol_fee_balance`. The `conditional_profit` is calculated as `spot_profit + pass_profit = 299 + 400 = 699`.
9. The amount of pMETA user received is calculated as `conditional_output + conditional_profit = 500 + 699 = 1199`.

The flaw appears in the calculation of `fail_profit` in Step 7. In Step 5, the protocol obtained 799 fMETA, after repaying the borrowed 500 fMETA, there should be 299 fMETA left. This remainder should have been added to `fail_profit`, but it was omitted.

Recommendation

Revise the calculation of `pass_profit` and `fail_profit` to correctly account for profits derived from token splitting.

Mitigation Review Log

Fixed in commit 6c37fc4ac13dce1336de6536b21dd1bbcb216e61.

4.6 Liquidity Provided During DAO Initialization Is Not Tracked in a Position

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When initializing a DAO, the creator may optionally provide initial liquidity in both base and quote tokens to the AMM pool. However, if the creator does provide liquidity, their position account is not created accordingly.

Impact

The added liquidity may effectively be donated to the AMM pool, since the creator cannot withdraw it due to the absence of a corresponding position account.

Recommendation

Ensure that a position account is created for the DAO creator if initial liquidity is provided.

Mitigation Review Log

Fixed in commit 5aad7ec65cc669b4cfda60f79d8d8a66e9d832e0, by disallowing liquidity provision during DAO initialization.



4.7 Incorrect FutarchyAmm Initialization When No Liquidity is Provided During DAO Creation

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When initializing a DAO, the creator may optionally provide initial liquidity in both base and quote tokens to the AMM pool.

```
180 if let Some(dao_creator_base_account) =  
    ⇒ ctx.accounts.dao_creator_base_account.as_ref() {  
181     token::transfer(  
182         CpiContext::new(  
183             ctx.accounts.token_program.to_account_info(),  
184             token::Transfer {  
185                 from: dao_creator_base_account.to_account_info(),  
186                 to:  
    ⇒ ctx.accounts.futarchy_amm_base_vault.to_account_info(),  
187                 authority: ctx.accounts.dao_creator.to_account_info(),  
188             },  
189         ),  
190         base_liquidity_to_lp,  
191     )?;  
192 }
```

[programs/autocrat/src/instructions/initialize_dao.rs#L180-L192](#)

However, the `futarchy_amm` is initialized incorrectly. The fields `quote_reserves`, `base_reserves`, and `total_liquidity` are always set to the provided parameters regardless of whether the token transfer above actually occurs.

```
229     futarchy_amm: FutarchyAmm {  
230         state: PoolState::Spot {  
231             spot: Pool {  
232                 quote_reserves: quote_liquidity_to_lp,  
233                 base_reserves: base_liquidity_to_lp,  
234                 ...  
235             },  
236         },  
237         total_liquidity: quote_liquidity_to_lp as u128 *  
    ⇒ 1_000_000_000,
```

[programs/autocrat/src/instructions/initialize_dao.rs#L229-L237](#)



Impact

If no initial liquidity is transferred to the AMM vaults by the DAO creator, but the reserves are still recorded as non-zero, the internal accounting will exceed the actual token balances. This mismatch may cause DoS conditions when users attempt swaps or liquidity withdrawals.

Recommendation

Adjust the initialization logic so that reserves and liquidity fields in `futarchy_amm` are set to non-zero values only if the corresponding token transfers succeed.

Mitigation Review Log

Fixed in commit `5aad7ec65cc669b4cfda60f79d8d8a66e9d832e0`, by disallowing liquidity provision during DAO initialization.

4.8 Missing Validation for `description_url` During Proposal Initialization

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

The `Proposal` account contains two fields with variable length: `description_url` and `stakers`.

```
25 #[derive(InitSpace)]
26 pub struct Proposal {
27     ...
28     #[max_len(40)]
29     pub description_url: String,
30     ...
31     /// Mapping of staker to amount staked (only used in Draft state)
32     #[max_len(100)]
33     pub stakers: Vec<StakerRecord>,
34 }
```

[programs/autocrat/src/state/proposal.rs#L25-L34](#)

Note that the `#[max_len(40)]` attribute in Anchor is not enforced at runtime. Therefore, it is necessary to manually validate the length of these fields whenever they are accessed or modified. Currently, no such checks are in place.



Impact

During proposal initialization, `description_url` can be set to a string longer than 40 characters. This may reduce the effective capacity of the `stakers` vector. In extreme cases, if `description_url` is set to a very long string (e.g., 4040 characters), no user would be able to stake, as there would be insufficient storage for even a single `StakerRecord`. Conversely, if `description_url` is shorter than 8 characters, the effective upper bound of `stakers` should be 101 instead of 100.

Recommendation

Add explicit runtime checks for:

- The length of `description_url` in the `InitializeProposal` instruction.
- The length of the `stakers` vector in the `StakeToProposal` instruction.

Mitigation Review Log

Fixed in commit 3fe4bc16b17e96b71672d968016dba911fd5ed60.

4.9 Malicious Proposals Can Block New Proposals in DAO

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Protocol Design

Description

The AMM `futarchy_amm` is bound to the DAO as a whole rather than to individual proposals. When a proposal is launched, the `futarchy_amm` must transition from the `Spot` state to the `Futarchy` state. Once the proposal is finalized, the state transitions back to `Spot`. This design implies that each DAO can only have one active proposal at any given time.

Impact

Since any user can create and launch a proposal in a DAO, a malicious user could initiate and launch a meaningless proposal to occupy the only available proposal slot. This would result in a DoS, preventing legitimate proposals from being launched.

Recommendation

Fixing this issue completely may require redesigning the binding relationship between the DAO and the AMM. As a workaround, the proposal creator could be required to lock a certain amount of LP tokens in the AMM until the proposal is finalized.



Mitigation Review Log

MetaDAO Team: It is okay with taking this risk since we have program upgrade authority and could address it if it became an issue.

4.10 AMM Position Account Cannot Be Closed

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Protocol Design

Description

The AMM position account (`amm_position`) tracks the liquidity provided by a user. Currently, this account cannot be closed, even after the user has withdrawn all their liquidity.

Impact

Users are unable to close their position account to reclaim the rent, which may negatively affect the user experience.

Recommendation

Allow the `amm_position` account to be closed if the user has withdrawn all their liquidity.

Mitigation Review Log

MetaDAO Team: We agree this would be a QoL improvement for users, we've added a todo for it.

4.11 Informational and Undetermined Issues

Missing Validation of `slots_per_proposal` When Updating DAO

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Data Validation

When update the DAO in `update_dao` IX, the provided parameter `slots_per_proposal` is not checked. It should be larger than `dao.twap_start_delay_slots` .

Fixed in commit 60e4fd730349d45a99648b9d2ad61f9a98446134.



Redundant Arbitrage Calculations in Trivial Cases

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Optimization

When performing arbitrage, there are cases where subsequent calculations are unnecessary and the function may return early:

- In `arbitrage_after_spot_swap`, if the `step_size` is zero or the updated `best_input_amount` is still zero, the function can return immediately.
- In `arbitrage_after_conditional_swap`, if the `step_size` is zero or the updated `best_arb_input_amount` is still zero, the function can also return early. If the `spot_profit` is negative, the function can return early as well. Additionally, if either `fail_remaining_from_step_1` or `pass_remaining_from_step_1` is zero, there is no need to calculate `fail_profit_from_remaining` or `pass_profit_from_remaining`, since they must also be zero.



5 Disclaimer

This report reflects the security status of the project as of the date of the audit. It is intended solely for informational purposes and should not be used as investment advice. Despite carrying out a comprehensive review and analysis of the relevant smart contracts, it is important to note that Offside Labs' services do not encompass an exhaustive security assessment. The primary objective of the audit is to identify potential security vulnerabilities to the best of the team's ability; however, this audit does not guarantee that the project is entirely immune to future risks.

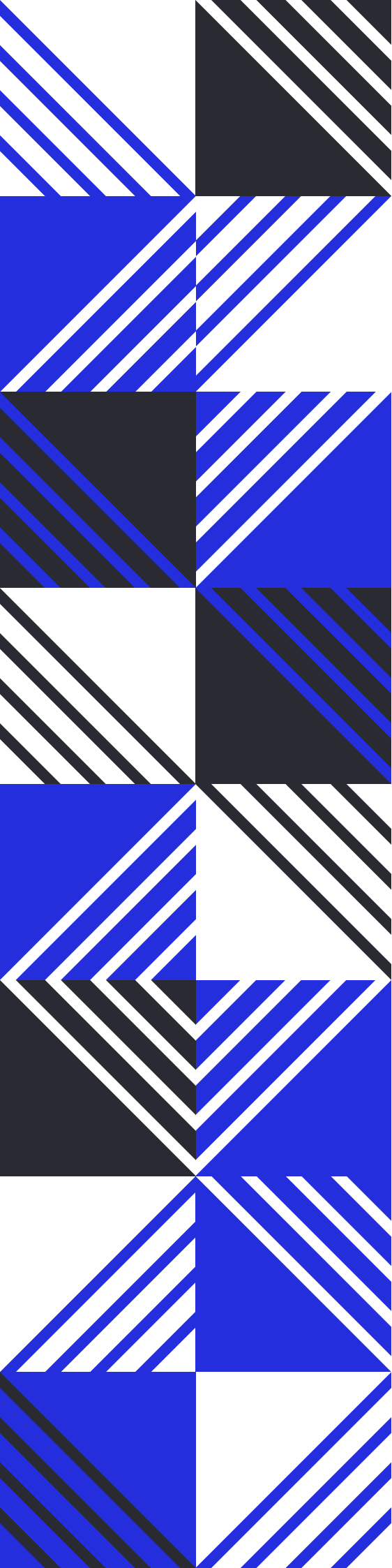
Offside Labs disclaims any liability for losses or damages resulting from the use of this report or from any future security breaches. The team strongly recommends that clients undertake multiple independent audits and implement a public bug bounty program to enhance the security of their smart contracts.

The audit is limited to the specific areas defined in Offside Labs' engagement and does not cover all potential risks or vulnerabilities. Security is an ongoing process, regular audits and monitoring are advised.

Please note: Offside Labs is not responsible for security issues stemming from developer errors or misconfigurations during contract deployment and does not assume liability for centralized governance risks within the project. The team is not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By utilizing this report, the client acknowledges the inherent limitations of the audit process and agrees that the firm shall not be held liable for any incidents that may occur after the completion of this audit.

This report should be considered null and void in case of any alteration.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs